Разработка модулей ядра Linux: Часть 3. Экспорт символов

Олег Цилюрик
преподаватель тренингового отделения
Global Logic

26.04.2012

Эта статья продолжает цикл статей, посвященный написанию модулей ядра Linux. В статье представлены приемы программирования, используемые для отладки и диагностики модулей ядра.

Введение

В этой статье рассматривается вопрос, касающийся экспорта символов из ядра (и модулей) и множества экспортируемых символов, составляющих API ядра, или, по аналогии с прикладным POSIX программированием, библиотеку средств, доступных программисту модуля.

Список экспортируемых символов

Далеко не все имена, перечисленные в /proc/kallsyms, являются экспортируемыми:

```
$ cat /proc/kallsyms | wc -l
69423
```

Какие из этих 100000 или около того имён (в зависимости от версии ядра) являются экспортируемыми и доступными для использования в модуле? Прежде всего, необходимо посмотреть, как выглядят имена, экспортируемые модулем **md1**. Чтобы исключить из имён слишком длинные пути рассматриваемый проект необходимо перенести в каталог **$HOME/TMP**:

Листинг 1. Символы, экспортируемые модулем

```
$ pwd
/home/olej/TMP
$ ls
Makefile  md1.c  md1.ko  md2.c  md2.ko  md3.c  md3.ko  md.h  md.hist  Module.symvers
$ cat Module.symvers
0x00000000 md1_data /home/olej/TMP/md1 EXPORT_SYMBOL
0x00000000 md1_proc /home/olej/TMP/md1 EXPORT_SYMBOL
```

© Copyright IBM Corporation 2012

В полученном выводе приведены экспортируемые имена, вместе с указанием типа экспорта: **EXPORT_SYMBOL** или **EXPORT_SYMBOL_GPL**. Точно так же можно найти символы, экспортируемые ядром и инсталлированные модулем системы:

Листинг 2. Символы, экспортируемые ядром

```
$ cat /lib/modules/`uname -r`/build/Module.symvers | grep 'printk'
0x00000000 __snd_printk sound/core/snd EXPORT_SYMBOL_GPL
...
0x00000000 printk vmlinux EXPORT_SYMBOL
0x00000000 drm_ut_debug_printk drivers/gpu/drm/drm EXPORT_SYMBOL
...
0x00000000 vprintk vmlinux EXPORT_SYMBOL
...
```

Здесь, как и в случае с модулем **md1**, используется тот же формат тот же:

```
имя – путь к экспортирующему модулю – тип экспорта
```

только в случае автономной сборки **md1** указывался абсолютный путь (**/home/olej/TMP/md1**), а здесь указывается путь относительно **/lib/modules/`uname -r` /build (drivers/gpu/drm/drm)**.

К этому моменту уже было показано, как написать и собрать собственный модуль, просмотреть важнейшие системные файлы, содержащие информацию о символах ядра, а также представлены несколько инструментов для анализа. И, хотя пока это только разрозненные примеры и этого уже достаточно для начала работы, но осталось рассмотреть первостепенные вопросы, перечисленные ниже.

Вывод диагностики модуля

Как было показано, для диагностического вывода из модуля использовался вызов **printk()**. Он настолько подобен по функциональности и формату общеизвестному методу из пользовательского пространства **printf()**, что не требует дополнительного описания. Это объясняется просто: и в том и в другом случае всю полезную работу по форматированию и подготовке вывода делают функции **sprintf()**, а **printk()** или **printf()** только перенаправляют сформированный вывод в нужное им место.

Примечание автора. Функция **sprintf()** - это две различных функции: одна находится в ядре, а другая в библиотеке **libc.so**, при этом у них совпадают сигнатуры и практически одинаковая функциональность.

Однако **printk()** все-таки отличается от стандартной функции **printf()**. Главное отличие состоит в том, что первому параметру **может** предшествовать (а может и не предшествовать, но тогда это просто означает, что используется значение по умолчанию) константа квалификатор, определяющая **уровень сообщений**. Определения констант для 8-ми уровней сообщений, записываемых в вызове **printk()** можно найти в файле **/lib/modules/`uname -r`/build/include/linux/kernel.h** (в программах на языке С коде этот заголовочный файл записывается как **<linux/kernel.h>**):

Листинг 3. Уровни диагностики

```
#define KERN_EMERG      "<0>"   /* использование системы невозможно          */
#define KERN_ALERT      "<1>"   /* действие должно быть выполнено немедленно */
#define KERN_CRIT       "<2>"   /* критическая ситуация                      */
#define KERN_ERR        "<3>"   /* сообщения об ошибках                      */
#define KERN_WARNING    "<4>"   /* предупреждающие сообщения                 */
#define KERN_NOTICE     "<5>"   /* ситуация нормальная, но требует внимания  */
#define KERN_INFO       "<6>"   /* информационные сообщения                  */
#define KERN_DEBUG      "<7>"   /* отладочные сообщения                      */
```

Эта предшествующая константа в вызове **printk()** не является отдельным параметром (не отделяется запятой), и (как видно из примеров) представляет собой символьную строку определённого вида, которая **конкатенируется** с первым параметром (являющимся, в общем случае, **форматной** строкой). Если данная константа отсутствует, то для этого сообщения устанавливается уровень вывода по умолчанию (часто это KERN_WARNING). Таким образом, следующие формы записи считаются эквивалентными:

```
printk( KERN_WARNING "string" );
printk( "<4>" "string" );
printk( "<4>string" );
printk( "string" );
```

Вызов **printk()** не выполняет непосредственно какой-либо вывод, а направляет выводимую строку демону системного журнала, который уже перезаписывает полученную строку на **текстовую консоль** и в файл системного журнала. При работе в **графической** системе X11, вывод **printk()** в терминал не попадет и останется только в системном журнале. При работе в текстовой консоли (куда можно оперативно переключаться комбинацией клавиш <Ctrl><Alt><Fi>, где Fi - функциональная клавиша i-го виртуального терминала) на экране отображается только вывод, выполняемый **printk()**, сообщений с уровнем **выше порогового значения**, установленного при запуске демона системного журнала (**klogd** или **rsyslog** или любого другого). Пороговое значение для информации, выводимой демоном журнала, устанавливается при его старте. Конфигурация уровня выводимых сообщений выходит за пределы данной публикации, тем более что эти способы различаются для различных программ демонов, используемых в системе (**klogd** или **rsyslog**). В любом случае, после изменения порогового значения демон нужно **перезапустить**.

Уровнем выводимой информации можно управлять через псевдофайл **/proc/sys/kernel/printk**:

```
$ cat /proc/sys/kernel/printk
3       4       1       7
```

Первая цифра в этой последовательности — это уровень чувствительности, все сообщения с более низкими уровнями (**KERN_EMERG, KERN_ALERT, KERN_CRIT**) ещё будут отображаться в консоли, а сообщения более высокого уровня – уже нет. Этот порог можно изменять, до следующей перезагрузки, динамически (правда, сделать это можно только в терминале от имени пользователя root, и, из-за перенаправления вывода, **sudo** здесь не поможет):

```
# echo 8 > /proc/sys/kernel/printk $ cat < /proc/sys/kernel/printk
8       4       1       7
```

В примерах, прикрепленных к этой статье, есть проект **log_level.tgz** для проверки и демонстрации порогов вывода сообщений, который после внесения изменений, показанных выше, выведет в **текстовую консоль**:

Листинг 4. Изменённый порог диагностики

```
# insmod log_level.ko
message level: <7>
message level: <6>
message level: <5>
message level: <4>
message level: <3>
message level: <2>
message level: <1>
insmod: error inserting 'log_level.ko': -1 Operation not permitted
```

Заключение

К данному моменту был полностью пройден весь процесс создания и анализа модуля и осталось только систематизировать стандартные ошибки, возникающие в процессе загрузки модуля. Этот вопрос будет рассмотрен в следующей статье, а затем мы вернемся к детальному и систематизированному рассмотрению вопросов архитектуры системы, модулей, их соотношений, и взаимодействием с приложениями из пользовательских адресных пространств.

Загрузка

Описание	Имя	Размер
	log_level.tgz	1KB

Об авторе

Олег Цилюрик



Олег Иванович Цилюрик, много лет был разработчиком программного обеспечения в крупных центрах разработки: ВНИИ РТ, НПО "Дельта", КБ ПМ. Последние годы работал над проектами в области промышленной автоматики, IP телефонии и коммуникаций. Автор нескольких книг. Преподаватель тренингового отделения международной софтверной компании Global Logic.

© Copyright IBM Corporation 2012
(www.ibm.com/legal/copytrade.shtml)
Торговые марки
(www.ibm.com/developerworks/ru/ibm/trademarks/)